

```

1  //-----CHAP 6 COLLECTION-II QB SOLUTION Q261 TO Q274 -----//
2
3  /* QB261
4  Write a Java program that takes a list of integers as input from the user and stores
   them in a PriorityQueue. The program should then remove and display the top three
   highest integers from the PriorityQueue.*/
5
6  import java.util.*;
7
8  public class TopThreeHighestIntegers {
9      public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         PriorityQueue<Integer> priorityQueue = new PriorityQueue<>(Collections.
            reverseOrder());
12
13         System.out.print("Enter the number of integers: ");
14         int numIntegers = scanner.nextInt();
15         scanner.nextLine(); // Consume the newline character left by nextInt()
16
17         for (int i = 0; i < numIntegers; i++) {
18             System.out.print("Enter integer " + (i + 1) + ": ");
19             int value = scanner.nextInt();
20             priorityQueue.add(value);
21         }
22
23         System.out.println("Top three highest integers:");
24
25         // Removing and displaying the top three highest integers
26         for (int i = 0; i < 3; i++) {
27             if (!priorityQueue.isEmpty()) {
28                 int highestInt = priorityQueue.poll();
29                 System.out.println(highestInt);
30             } else {
31                 System.out.println("Not enough integers in the PriorityQueue.");
32                 break;
33             }
34         }
35
36         scanner.close();
37     }
38 }
39
40 //-----
41
42 /* QB262
43 Sports tournament organizer: Write a program that simulates a sports tournament using a
   PriorityQueue. The program should allow the user to input team names and their win-loss
   records. The program should then prioritize teams based on their win-loss records and
   add them to the PriorityQueue. When its time for the next match, the program should
   remove the two highest priority teams from the PriorityQueue and display their names.*/
44
45 import java.util.*;
46
47 class Team {
48     String name;
49     int wins;
50     int losses;
51
52     public Team(String name, int wins, int losses) {
53         this.name = name;
54         this.wins = wins;
55         this.losses = losses;
56     }
57
58     public int getWinLossRatio() {
59         return wins - losses;
60     }
61 }
62

```

```

63     @Override
64     public String toString() {
65         return name;
66     }
67 }
68
69 public class SportsTournamentSimulator {
70     public static void main(String[] args) {
71         Scanner scanner = new Scanner(System.in);
72         PriorityQueue<Team> teamsQueue = new PriorityQueue<>(Comparator.comparing(Team::
getWinLossRatio).reversed());
73
74         // Input team names and win-loss records
75         System.out.println("Enter the number of teams: ");
76         int numTeams = scanner.nextInt();
77         scanner.nextLine(); // Consume the newline character after reading the integer.
78
79         for (int i = 0; i < numTeams; i++) {
80             System.out.println("Enter team name: ");
81             String name = scanner.nextLine();
82             System.out.println("Enter number of wins: ");
83             int wins = scanner.nextInt();
84             System.out.println("Enter number of losses: ");
85             int losses = scanner.nextInt();
86             scanner.nextLine(); // Consume the newline character after reading the
integer.
87
88             Team team = new Team(name, wins, losses);
89             teamsQueue.add(team);
90         }
91
92         // Display the next match
93         System.out.println("Next match:");
94         if (!teamsQueue.isEmpty()) {
95             Team team1 = teamsQueue.poll();
96             Team team2 = teamsQueue.poll();
97             System.out.println(team1 + " vs. " + team2);
98         } else {
99             System.out.println("Not enough teams to start the match.");
100         }
101     }
102 }
103
104 //-----
105
106 /*
107  * QB 263
108  * Write a program that simulates a queue of customers at a coffee
109  shop using an ArrayDeque. The program should allow the user to
110  perform the following actions:
111  Add a new customer to the back of the queue
112  Serve the next customer in the queue (i.e. remove the customer
113  from the front of the queue)
114  View the current queue of customers
115  The program should continue to prompt the user for actions until
116  they choose to quit.
117
118  */
119 import java.util.*;
120
121 public class QB263 {
122     public static void main(String[] args) {
123         ArrayDeque<String> customerQueue = new ArrayDeque<>();
124         Scanner sc = new Scanner(System.in);
125         int ch;
126         do{
127             System.out.println("\nMenu:");
128             System.out.println("1. Add new customer to the back of the queue");

```

```

130         System.out.println("2. Serve the next customer");
131         System.out.println("3. View the current queue");
132         System.out.println("4. Quit");
133
134         System.out.print("Enter your choice: ");
135         ch = sc.nextInt();
136         sc.nextLine(); // Clear the newline character
137
138         switch (ch) {
139             case 1:
140                 System.out.print("Enter customer name: ");
141                 String customer = sc.nextLine();
142                 customerQueue.addLast(customer);
143                 System.out.println(customer + " has been added to the queue.");
144                 break;
145             case 2:
146                 if (!customerQueue.isEmpty()) {
147                     String servedCustomer = customerQueue.pollFirst();
148                     System.out.println("Serving " + servedCustomer + ".");
149                 } else {
150                     System.out.println("Queue is empty. No customers to serve.");
151                 }
152                 break;
153             case 3:
154                 if (!customerQueue.isEmpty()) {
155                     System.out.println("Current queue of customers:");
156                     for (String name : customerQueue) {
157                         System.out.println("- " + name);
158                     }
159                 } else {
160                     System.out.println("Queue is empty. No customers in the queue.");
161                 }
162                 break;
163             case 4:
164                 System.out.println("Exiting coffee shop queue simulation.");
165
166             default:
167                 System.out.println("Invalid choice. Please try again.");
168         }
169         }while(ch!=4);
170     }
171 }
172
173 /*
174  * QB 264
175  *
176  * Music player playlist: Write a program that simulates a music
177  * player playlist using an ArrayDeque. The program should allow
178  * the user to add songs to the playlist, play the next or previous
179  * song in the playlist, and shuffle the playlist. The program should
180  * keep track of the songs in the playlist using an ArrayDeque,
181  * where each element in the deque represents a song. When the
182  * user adds a song, you can use the addLast method to add it to
183  * the end of the playlist. When the user plays the next or previous
184  * song, you can use the removeFirst or removeLast method,
185  * respectively, to remove the current song from the front or back
186  * of the deque and add it to the end or beginning of the deque.
187  * When the user shuffles the playlist, you can use the shuffle
188  * method to randomly reorder the songs in the deque.
189  *
190  */
191
192
193 import java.util.ArrayDeque;
194 import java.util.ArrayList;
195 import java.util.Collections;
196 import java.util.List;
197 import java.util.Scanner;
198

```

```

200 public class QB264 {
201
202     public static void main(String[] args) {
203         Scanner scanner = new Scanner(System.in);
204         ArrayDeque<String> playlist = new ArrayDeque<>();
205
206         System.out.println("Music Player Playlist Simulator");
207
208         while (true) {
209             System.out.println("\nMenu:");
210             System.out.println("1. Add song to playlist");
211             System.out.println("2. Play next song");
212             System.out.println("3. Play previous song");
213             System.out.println("4. Shuffle playlist");
214             System.out.println("5. Display current playlist");
215             System.out.println("6. Exit");
216
217             System.out.print("Enter your choice: ");
218             int choice = Integer.parseInt(scanner.nextLine());
219
220             switch (choice) {
221                 case 1:
222                     System.out.print("Enter song name: ");
223                     String song = scanner.nextLine();
224                     playlist.addLast(song);
225                     System.out.println(song + " added to the playlist.");
226                     break;
227                 case 2:
228                     if (!playlist.isEmpty()) {
229                         String nextSong = playlist.removeFirst();
230                         System.out.println("Playing: " + nextSong);
231                         playlist.addLast(nextSong);
232                     } else {
233                         System.out.println("Playlist is empty.");
234                     }
235                     break;
236                 case 3:
237                     if (!playlist.isEmpty()) {
238                         String previousSong = playlist.removeLast();
239                         System.out.println("Playing: " + previousSong);
240                         playlist.addFirst(previousSong);
241                     } else {
242                         System.out.println("Playlist is empty.");
243                     }
244                     break;
245                 case 4:
246                     if (!playlist.isEmpty()) {
247                         ArrayList<String> al = new ArrayList<>(playlist);
248                         Collections.shuffle(al);
249                         playlist = new ArrayDeque<>(al);
250
251                         System.out.println("Playlist shuffled.");
252                     } else {
253                         System.out.println("Playlist is empty.");
254                     }
255                     break;
256                 case 5:
257                     if (!playlist.isEmpty()) {
258                         System.out.println("Current Playlist: " + playlist);
259                     } else {
260                         System.out.println("Playlist is empty.");
261                     }
262                     break;
263                 case 6:
264                     System.out.println("Exiting Music Player Playlist Simulator.");
265                     scanner.close();
266                     System.exit(0);
267                     break;

```

```

268         default:
269             System.out.println("Invalid choice. Please try again.");
270     }
271 }
272 }
273 }
274
275 // QB265
276 /*Make the system as User interactive and give choices to user.
277 Web browser history: Write a program that simulates a web browser history using an
ArrayDeque. The program should allow the user to navigate back and forward through their
history of visited web pages, similar to how a web browser works. The program should
keep track of the URLs of visited web pages using an ArrayDeque, where each element in
the deque represents a visited web page. When the user navigates to a new web page, you
can push the URL onto the deque. When the user navigates back or forward, you can pop
URLs off the front or back of the deque, respectively.*/
278
279 import java.util.ArrayDeque;
280 import java.util.Deque;
281 import java.util.Scanner;
282
283 public class WebBrowserHistory {
284     static Deque<String> history = new ArrayDeque<>();
285
286     public static void main(String[] args) {
287         Scanner scanner = new Scanner(System.in);
288         int choice;
289
290         do {
291             printMenu();
292             choice = scanner.nextInt();
293             scanner.nextLine(); // Consume the newline character after reading the
integer
294
295             switch (choice) {
296                 case 1:
297                     System.out.print("Enter the URL to visit: ");
298                     String url = scanner.nextLine();
299                     navigateTo(url);
300                     break;
301                 case 2:
302                     navigateBack();
303                     break;
304                 case 3:
305                     navigateForward();
306                     break;
307                 case 4:
308                     showHistory();
309                     break;
310                 case 5:
311                     System.out.println("Exiting the web browser.");
312                     break;
313                 default:
314                     System.out.println("Invalid choice. Please try again.");
315             }
316         } while (choice != 5);
317
318         scanner.close();
319     }
320
321     static void printMenu() {
322         System.out.println("===== Web Browser History =====");
323         System.out.println("1. Visit a new web page");
324         System.out.println("2. Navigate back");
325         System.out.println("3. Navigate forward");
326         System.out.println("4. Show history");
327         System.out.println("5. Exit");
328         System.out.print("Enter your choice: ");
329     }

```

```

330
331     static void navigateTo(String url) {
332         history.addLast(url);
333         System.out.println("Navigated to: " + url);
334     }
335
336     static void navigateBack() {
337         if (history.isEmpty()) {
338             System.out.println("No history to navigate back.");
339         } else {
340             String url = history.pollLast();
341             System.out.println("Navigated back to: " + url);
342         }
343     }
344
345     static void navigateForward() {
346         if (history.isEmpty()) {
347             System.out.println("No history to navigate forward.");
348         } else {
349             String url = history.pollFirst();
350             System.out.println("Navigated forward to: " + url);
351         }
352     }
353
354     static void showHistory() {
355         if (history.isEmpty()) {
356             System.out.println("No history available.");
357         } else {
358             System.out.println("Web Browser History:");
359             int index = 1;
360             for (String url : history) {
361                 System.out.println(index++ + ". " + url);
362             }
363         }
364     }
365 }
366
367 //-----
368
369
370 /* QB266
371  *
372  * In my restaurant I used to manage it well as per the order. I never
373  want to skip any order from the customer. So I prepare a rule:
374  When I get the order I add it in the last of my cook queue. And
375  when the order is ready I used to pick it up from the first of the
376  cook queue.
377  So, write a java program with class Restaurant. Create a queue
378  cook which contains item names. Ask user weather he wants to
379  Order or take food.
380
381  If user press 1 then ask for the name of the
382  item and it should be added in the cook queue.
383
384  If user press 2 then The first item from the queue should be
385  removed from the queue.
386
387  If user press 3 then he can see the items of the queue. If user
388  press 4 then he should move out from the Restaurant.
389
390  */
391
392 import java.util.ArrayDeque;
393 import java.util.*;
394
395 class Restaurant{
396
397     ArrayDeque<String> cook;
398

```

```

399     Restaurant() {
400         cook = new ArrayDeque<>();
401     }
402
403 }
404 class QB266 {
405
406     public static void main(String[] args) {
407         Scanner sc = new Scanner(System.in);
408         Restaurant r = new Restaurant();
409         int ch;
410         do{
411             System.out.println("Enter Your Choice 1 To What to Order? \n 2 for Serve the
412             order \n 3. view Order \n 4. Move out from restaurant");
413             ch = sc.nextInt();
414             sc.nextLine();
415             switch(ch){
416                 case 1:
417                     System.out.println("Enter Item to Order");
418                     String item = sc.nextLine();
419                     r.cook.addLast(item);
420                     System.out.println("Your order for item "+item+" has been placed..."
421                     );
422                     break;
423                 case 2:
424                     if(!r.cook.isEmpty()){
425                         String servingItem = r.cook.pollFirst();
426                         System.out.println("Serving item "+servingItem+" to customer...."
427                         );
428                     }else{
429                         System.out.println("NO ORDER TO SERVE");
430                     }
431                     break;
432                 case 3:
433                     if(!r.cook.isEmpty()){
434                         System.out.println("----- LIST OF ITEMS IN QUEUE TO BE COOKED
435                         -----");
436                         int i = 1;
437                         for(String s:r.cook){
438                             System.out.println("Item "+i+" = "+s);
439                             i++;
440                         }
441                     }else{
442                         System.out.println("NO ORDER TO VIEW IN QUEUE");
443                     }
444                     break;
445                 case 4:
446                     System.out.println("THANK YOU... VISIT AGAIN");
447                     break;
448             }
449         }while(ch!=4);
450     }
451 }
452
453 }
454 // QB267
455 /*Write a java program that reads in a list of integers from the user
456 and stores them in a HashSet. The program should then compute
457 the sum of all unique integers in the HashSet and output the
458 result to the user. If the user enters the same integer multiple
459 times, it should only be counted once in the sum.
460 */
461
462 import java.util.*;
463

```

```

464 public class QB267 {
465     public static void main(String[] args) {
466         HashSet<Integer> uniqueValues = new HashSet<>();
467         Scanner sc = new Scanner(System.in);
468         while(true){
469             System.out.println("Enter Integer Number ");
470             int no = sc.nextInt();
471             sc.nextLine();
472             System.out.println("DO YOU WANT TO CONTINUE ? YES OR NO");
473             uniqueValues.add(no);
474
475             String choice = sc.nextLine();
476             if(choice.equalsIgnoreCase("NO")){
477                 break;
478             }
479         }
480         int sum = 0;
481         for(Integer i:uniqueValues){
482             sum = sum + i;
483         }
484         System.out.println("SUM = "+sum);
485     }
486 }
487
488 // QB268
489 /*Write a program that reads in a list of words from the user and
490 stores them in a HashSet. The program should then prompt the
491 user for a prefix and output all the words in the set that start with
492 that prefix.
493 */
494
495 import java.util.*;
496
497 public class QB268 {
498     public static void main(String[] args) {
499         HashSet<String> words = new HashSet<>();
500         Scanner sc = new Scanner(System.in);
501
502         // Read words from user and stores them in to HashSet
503
504         System.out.println("-----");
505         while (true) {
506             System.out.print("Enter Word ");
507             String w = sc.nextLine();
508             words.add(w);
509             System.out.println("DO YOU WANT TO CONTINUE ? YES OR NO");
510
511             String choice = sc.nextLine();
512             if(choice.equalsIgnoreCase("NO")){
513                 break;
514             }
515         }
516
517         System.out.println("Enter prefix for the word that you want to search from
518 HashSet ? ");
519         String prefix = sc.next();
520         for(String s : words){
521             if(s.startsWith(prefix)){
522                 System.out.println("----> "+s );
523             }
524         }
525     }
526
527 /* QB269
528  * Write a program that reads in a list of names and corresponding
529 phone numbers from the user, and stores them in a HashMap.
530 The program should then prompt the user for a name and output
531 the corresponding phone number, or a message indicating that

```

```

532 the name is not in the map.
533
534 */
535 import java.util.HashMap;
536 import java.util.*;
537
538 public class QB269 {
539     public static void main(String[] args) {
540         HashMap<String,String> phonebook = new HashMap<>();
541         Scanner sc = new Scanner(System.in);
542         String name, number;
543         while(true){
544             System.out.println("Enter Name");
545             name = sc.nextLine();
546
547             System.out.println("Enter Number");
548             number = sc.nextLine();
549             phonebook.put(name, number);
550             System.out.println("Do you want to add Data ? Enter Yes or No");
551             String choice = sc.nextLine();
552             if(choice.equalsIgnoreCase("NO")){
553                 break;
554             }
555         }
556
557         System.out.println("Enter the name you want to search from phonebook ?");
558         String searchName = sc.nextLine();
559         String searchNumber = phonebook.get(searchName);
560         if(searchNumber!=null){
561             System.out.println("Phone Number of "+searchName+" = "+searchNumber);
562         }
563         else{
564             System.out.println("Name is not there in phonebook");
565         }
566     }
567 }
568
569
570
571 /* QB270
572  * Write a program that reads in a list of stock prices for a company
573  and stores them in a HashMap, where the key is the date and the
574  value is the price. The program should then compute the average
575  price for the entire period and output it to the user.
576
577  */
578 import java.util.*;
579
580
581 public class QB270 {
582     public static void main(String[] args) {
583         HashMap<String, Double> stockPrices = new HashMap<>();
584         Scanner sc = new Scanner(System.in);
585
586         // Read stock prices and dates from the user and store them in the HashMap
587         System.out.println("Enter stock prices and dates");
588         while (true) {
589             System.out.print("Date (format: YYYY-MM-DD): ");
590             String date = sc.nextLine();
591             System.out.print("Price: ");
592
593             double price =sc.nextDouble();
594             sc.nextLine();
595
596             stockPrices.put(date, price);
597
598             System.out.println("DO YOU WANT TO CONTINUE ? YES OR NO");
599
600             String choice = sc.nextLine();

```

```

601         if(choice.equalsIgnoreCase("NO")){
602             break;
603         }
604     }
605 }
606
607 // Calculate the average stock price for the entire period
608 double total = 0.0;
609 for (double price : stockPrices.values()) {
610     total = total + price;
611 }
612 double average = total / stockPrices.size();
613
614 // Output the result
615 System.out.println("Average stock price for the entire period: " + average);
616
617 }
618 }
619
620
621 /* QB271
622  * Write a program that reads in a list of words from a file and
623  * stores them in a HashMap, along with their frequency (i.e. how
624  * many times they appear in the file). The program should then
625  * prompt the user for a word and output the corresponding
626  * frequency, or a message indicating that the word is not in the
627  * map.
628  */
629
630 import java.util.*;
631 import java.io.*;
632 public class QB271 {
633     public static void main(String[] args) throws Exception {
634         FileReader fr = new FileReader("inputData.txt");
635         BufferedReader br = new BufferedReader(fr);
636         HashMap<String,Integer> wordFrequencyCount = new HashMap<>();
637         String line="";
638         while((line = br.readLine())!=null){
639             String words[] = line.split(" ");
640             for(String w:words){
641                 if(wordFrequencyCount.get(w)==null){
642                     wordFrequencyCount.put(w,1);
643                 }
644                 else{
645                     wordFrequencyCount.put(w, (wordFrequencyCount.get(w)+1));
646                 }
647                 //wordFrequencyCount.put(w, wordFrequencyCount.getOrDefault(w, 0)+1); //
648                 // this method is not in syllabus so can not use it .
649             }
650         }
651         for(Map.Entry e: wordFrequencyCount.entrySet()){
652             System.out.println("Count for: "+e.getKey() + " is = "+ e.getValue());
653         }
654         br.close();
655         fr.close();
656     }
657 }
658
659 }
660
661 /* QB272
662  * Write a Java program that takes in a list of student names and their grades from the
663  * user and stores them in a HashTable. The program should then prompt the user for a
664  * student name and output their grade. If the student name is not found in the HashTable,
665  * the program should output an error message.*/
666
667 import java.util.Hashtable;
668 import java.util.Scanner;

```

```

666
667 public class StudentGradeLookup {
668     public static void main(String[] args) {
669         Scanner scanner = new Scanner(System.in);
670         Hashtable<String, Integer> studentGrades = new Hashtable<>();
671
672         // Input student names and grades
673         System.out.print("Enter the number of students: ");
674         int numStudents = scanner.nextInt();
675         scanner.nextLine(); // Consume the newline character
676
677         for (int i = 0; i < numStudents; i++) {
678             System.out.print("Enter student name: ");
679             String name = scanner.nextLine();
680
681             System.out.print("Enter student grade: ");
682             int grade = scanner.nextInt();
683             scanner.nextLine(); // Consume the newline character
684
685             studentGrades.put(name, grade);
686         }
687
688         // Lookup student grade
689         System.out.print("Enter student name to lookup grade: ");
690         String nameToLookup = scanner.nextLine();
691
692         Integer grade = studentGrades.get(nameToLookup);
693
694         if (grade != null) {
695             System.out.println("Grade for " + nameToLookup + ": " + grade);
696         } else {
697             System.out.println("Error: Student not found.");
698         }
699
700         scanner.close();
701     }
702 }
703
704 //-----
705
706 /* QB273
707     Write a Java program that implements a HashTable to store a dictionary of words and
       their definitions. The program should prompt the user for a word and output its
       definition. If the word is not found in the HashTable, the program should ask the
       user if they would like to add the word and its definition to the HashTable.
708 The program should use the following methods of the HashTable class:
709 put(K key, V value): Adds a key-value pair to the HashTable. In this program, it should
       be used to add new words and their definitions.
710 get(Object key): Retrieves the value associated with the given key from the HashTable.
       In this program, it should be used to retrieve the definition of a word.
711 containsKey(Object key): Returns true if the HashTable contains a mapping for the
       specified key. In this program, it should be used to check if a word is already in the
       HashTable.
712 keySet(): Returns a Set of all keys in the HashTable. In this program, it should be used
       to print out a list of all words in the dictionary.*/
713
714 import java.util.Hashtable;
715 import java.util.Scanner;
716 import java.util.Set;
717
718 public class DictionaryApp {
719     public static void main(String[] args) {
720         Hashtable<String, String> dictionary = new Hashtable<>();
721         Scanner scanner = new Scanner(System.in);
722
723         // Populate the dictionary with some initial words and definitions
724         dictionary.put("apple", "A fruit that is typically red or green.");
725         dictionary.put("banana", "A long, curved fruit with a yellow skin.");
726         dictionary.put("cat", "A small domesticated carnivorous mammal.");

```

```

727 while (true) {
728     System.out.print("Enter a word to get its definition (or type 'exit' to
729 quit): ");
730     String inputWord = scanner.nextLine();
731
732     if (inputWord.equalsIgnoreCase("exit")) {
733         System.out.println("Goodbye!");
734         break;
735     }
736
737     if (dictionary.containsKey(inputWord)) {
738         String definition = dictionary.get(inputWord);
739         System.out.println("Definition: " + definition);
740     } else {
741         System.out.println("Word not found in the dictionary.");
742
743         System.out.print("Would you like to add it? (yes/no): ");
744         String addWord = scanner.nextLine();
745
746         if (addWord.equalsIgnoreCase("yes")) {
747             System.out.print("Enter the definition of the word: ");
748             String definition = scanner.nextLine();
749             dictionary.put(inputWord, definition);
750             System.out.println("Word added to the dictionary.");
751         } else {
752             System.out.println("Word was not added to the dictionary.");
753         }
754     }
755 }
756
757 // Print out the list of all words in the dictionary
758 Set<String> words = dictionary.keySet();
759 System.out.println("Words in the dictionary:");
760 for (String word : words) {
761     System.out.println(word);
762 }
763
764 scanner.close();
765 }
766 }
767
768 //-----
769
770 /*QB274
771 Write a Java program that creates a HashTable to store information about books in a
772 library. The HashTable should use the book titles as keys and store information about
773 each book, such as the author, publisher, and year of publication. The program should
774 provide the following functionality:
775 Add a book: The program should prompt the user for a book title, author, publisher, and
776 year of publication, and add the book to the HashTable.
777 Remove a book: The program should prompt the user for a book title and remove the
778 corresponding entry from the HashTable.
779 Search for a book: The program should prompt the user for a book title and output the
780 corresponding information stored in the HashTable, such as the author, publisher, and
781 year of publication. If the book title is not found in the HashTable, the program should
782 o
783 List all books: The program should list all books in the HashTable, along with their
784 corresponding information.
785 To accomplish these tasks, you can use the following HashTable methods:
786 put(key, value): Inserts a key-value pair into the HashTable.
787 remove(key): Removes a key-value pair from the HashTable.
788 get(key): Returns the value corresponding to a given key in the HashTable.
789 containsKey(key): Returns true if the HashTable contains a given key.
790 keySet(): Returns a Set of all the keys in the HashTable."*/
791
792 import java.util.Hashtable;
793 import java.util.Scanner;
794 import java.util.Set;

```

```

786
787 public class LibraryManagement {
788     private Hashtable<String, Book> library;
789
790     public LibraryManagement() {
791         library = new Hashtable<>();
792     }
793
794     private class Book {
795         String title;
796         String author;
797         String publisher;
798         int year;
799
800         public Book(String title, String author, String publisher, int year) {
801             this.title = title;
802             this.author = author;
803             this.publisher = publisher;
804             this.year = year;
805         }
806     }
807
808     public void addBook() {
809         Scanner scanner = new Scanner(System.in);
810         System.out.print("Enter book title: ");
811         String title = scanner.nextLine();
812         System.out.print("Enter author: ");
813         String author = scanner.nextLine();
814         System.out.print("Enter publisher: ");
815         String publisher = scanner.nextLine();
816         System.out.print("Enter year of publication: ");
817         int year = scanner.nextInt();
818
819         Book book = new Book(title, author, publisher, year);
820         library.put(title, book);
821
822         System.out.println("Book added successfully!");
823     }
824
825     public void removeBook() {
826         Scanner scanner = new Scanner(System.in);
827         System.out.print("Enter book title to remove: ");
828         String title = scanner.nextLine();
829
830         if (library.containsKey(title)) {
831             library.remove(title);
832             System.out.println("Book removed successfully!");
833         } else {
834             System.out.println("Book not found in the library.");
835         }
836     }
837
838     public void searchBook() {
839         Scanner scanner = new Scanner(System.in);
840         System.out.print("Enter book title to search: ");
841         String title = scanner.nextLine();
842
843         if (library.containsKey(title)) {
844             Book book = library.get(title);
845             System.out.println("Title: " + book.title);
846             System.out.println("Author: " + book.author);
847             System.out.println("Publisher: " + book.publisher);
848             System.out.println("Year of Publication: " + book.year);
849         } else {
850             System.out.println("Book not found in the library.");
851         }
852     }
853
854     public void listAllBooks() {

```

```

855     if (library.isEmpty()) {
856         System.out.println("The library is empty.");
857     } else {
858         System.out.println("Listing all books in the library:");
859         Set<String> titles = library.keySet();
860         for (String title : titles) {
861             Book book = library.get(title);
862             System.out.println("Title: " + book.title);
863             System.out.println("Author: " + book.author);
864             System.out.println("Publisher: " + book.publisher);
865             System.out.println("Year of Publication: " + book.year);
866             System.out.println("-----");
867         }
868     }
869 }

870
871 public static void main(String[] args) {
872     LibraryManagement libraryManagement = new LibraryManagement();
873     Scanner scanner = new Scanner(System.in);
874
875     while (true) {
876         System.out.println("\nLibrary Management System");
877         System.out.println("1. Add a book");
878         System.out.println("2. Remove a book");
879         System.out.println("3. Search for a book");
880         System.out.println("4. List all books");
881         System.out.println("5. Exit");
882         System.out.print("Enter your choice: ");
883         int choice = scanner.nextInt();
884         scanner.nextLine(); // Consume newline character
885
886         switch (choice) {
887             case 1:
888                 libraryManagement.addBook();
889                 break;
890             case 2:
891                 libraryManagement.removeBook();
892                 break;
893             case 3:
894                 libraryManagement.searchBook();
895                 break;
896             case 4:
897                 libraryManagement.listAllBooks();
898                 break;
899             case 5:
900                 System.out.println("Exiting the program.");
901                 System.exit(0);
902                 break;
903             default:
904                 System.out.println("Invalid choice. Please try again.");
905         }
906     }
907 }
908
909
910 /*-----*/
911
912 //-----CHAP 7 FILE IO QB SOLUTION Q298 TO Q307 -----//
913
914 /*QB 298
915 Write a java program which read source file and destination file name and copy the
916 content of source file to destination file using InputStream class. */
917 import java.io.*;
918
919 public class FileCopyUsingInputStream {
920     public static void main(String[] args) {
921         try (BufferedReader reader = new BufferedReader(new InputStreamReader(System.in))) {
922             System.out.print("Enter the source file name: ");

```

```

922         String sourceFileName = reader.readLine();
923
924         System.out.print("Enter the destination file name: ");
925         String destinationFileName = reader.readLine();
926
927         copyFileUsingInputStream(sourceFileName, destinationFileName);
928
929         System.out.println("File copied successfully.");
930     } catch (IOException e) {
931         System.out.println("Error: " + e.getMessage());
932     }
933 }
934
935 public static void copyFileUsingInputStream(String sourceFileName, String
destinationFileName) throws IOException {
936     try (InputStream inputStream = new FileInputStream(sourceFileName);
937         OutputStream outputStream = new FileOutputStream(destinationFileName)) {
938
939         byte[] buffer = new byte[1024];
940         int bytesRead;
941
942         while ((bytesRead = inputStream.read(buffer)) != -1) {
943             outputStream.write(buffer, 0, bytesRead);
944         }
945     }
946 }
947 }
948

```

The program will prompt you to enter the source file name and destination file name. After you provide the names, it will copy the content of the source file to the destination file using the `InputStream` class.

```

950
951 //-----
952
953 /* QB 299
954 Write a java program which read numbers from number.txt file and store even number to
even.txt and odd number into odd.txt file */
955
956
957 class NumberSorter
958 {
959     public static void main(String[] args) throws Exception
960     {
961
962         BufferedReader reader = new BufferedReader(new FileReader("number.txt"));
963         BufferedWriter evenWriter = new BufferedWriter(new FileWriter("even.txt"));
964         BufferedWriter oddWriter = new BufferedWriter(new FileWriter("odd.txt"));
965
966         String line = reader.readLine();
967         while (line != null)
968         {
969             int number = Integer.parseInt(line);
970             if (number % 2 == 0)
971             {
972                 evenWriter.write(Integer.toString(number));
973                 evenWriter.newLine();
974                 line = reader.readLine();
975             } else {
976                 oddWriter.write(Integer.toString(number));
977                 oddWriter.newLine();
978                 line = reader.readLine();
979             }
980         }
981
982         System.out.println("Numbers sorted and saved successfully!");
983         evenWriter.close();
984         oddWriter.close();
985
986     }

```

```

987 }
988 //-----
989
990 /* QB 300
991 Read employee salary and calculate the income tax based on 10% of income and store it in
992 tax.txt file for five different employees. */
993
994 import java.io.FileWriter;
995 import java.io.IOException;
996 import java.io.BufferedWriter;
997 import java.util.Scanner;
998
999 public class IncomeTaxCalculator {
1000     public static void main(String[] args) {
1001         try {
1002             FileWriter fileWriter = new FileWriter("tax.txt");
1003             BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
1004
1005             Scanner scanner = new Scanner(System.in);
1006
1007             for (int i = 1; i <= 5; i++) {
1008                 System.out.print("Enter salary for Employee " + i + ": ");
1009                 double salary = scanner.nextDouble();
1010
1011                 double tax = calculateIncomeTax(salary);
1012                 bufferedWriter.println("Employee " + i + ": " + tax);
1013             }
1014
1015             bufferedWriter.close();
1016             System.out.println("Income tax calculated and saved to tax.txt.");
1017         } catch (IOException e) {
1018             System.out.println("Error while writing to the file: " + e.getMessage());
1019         }
1020
1021         static double calculateIncomeTax(double salary) {
1022             double taxRate = 0.10; // 10% tax rate
1023             return salary * taxRate;
1024         }
1025     }
1026
1027 //-----
1028
1029 /* Q 301
1030 Write a program that counts the no. of words in a text file. The file name is passed as a
1031 command line argument. The program should check whether the file exists or not. The
1032 words in the file are separated by white space characters */
1033 import java.io.*;
1034
1035 class WordCounter {
1036     public static void main(String[] args)
1037     {
1038         if (args.length != 1) {
1039             System.out.println("Please provide the file name as a command line argument."
1040 );
1041             return;
1042         }
1043
1044         try
1045         {
1046             String fileName = args[0];
1047
1048             BufferedReader reader = new BufferedReader(new FileReader(fileName));
1049
1050             int wordCount = 0;
1051             String line = reader.readLine();
1052             while (line != null)
1053             {
1054                 String[] words = line.split(" ");

```

```

1054         wordCount += words.length;
1055         line = reader.readLine();
1056     }
1057     System.out.println("Number of words in the file: " + wordCount);
1058 }
1059 catch (FileNotFoundException e)
1060 {
1061     System.out.println("The file does not exist.");
1062 }
1063 catch (IOException e)
1064 {
1065     System.out.println("An error occurred while reading the file.");
1066 }
1067 }
1068 }
1069 //-----
1070
1071 /* QB 302    --- OR ---
1072 /* QB 303
1073 Write a program to check that whether the name given from command line is file or not?
1074 If it is a file then print the size of file and if it is directory then it should
1075 display the name of all files in it.    */
1076
1077 import java.io.File;
1078
1079 class FileTypeChecker
1080 {
1081     public static void main(String[] args)
1082     {
1083         if (args.length != 1)
1084         {
1085             System.out.println("Please provide the file or directory name as a command
1086             line argument.");
1087             return;
1088         }
1089         String path = args[0];
1090         File file = new File(path);
1091         if (file.isFile())
1092         {
1093             System.out.println("The given path is a file.");
1094             System.out.println("File size: " + file.length());
1095         }
1096         else if (file.isDirectory())
1097         {
1098             System.out.println("The given path is a directory.");
1099             System.out.println("Files in the directory:");
1100             String files[] = file.list();
1101             for (String f : files)
1102             {
1103                 System.out.println(f);
1104             }
1105         }
1106         else
1107         {
1108             System.out.println("The given path does not exist or is neither a file nor a
1109             directory.");
1110         }
1111     }
1112 }
1113
1114
1115 /*QB 304
1116 Write a program that reads file name from user, through command line argument and
1117 displays/reads content of the text file on console.*/
1118

```

```

1119 import java.io.*;
1120 class FileContentReader
1121 {
1122     public static void main(String[] args)
1123     {
1124         if (args.length != 1)
1125         {
1126             System.out.println("Please provide the file name as a command line argument."
1127             );
1128             return;
1129         }
1130         String fileName = args[0];
1131
1132         try
1133         {
1134             BufferedReader reader = new BufferedReader(new FileReader(fileName));
1135
1136             String line= reader.readLine();
1137             while ( line != null)
1138             {
1139                 System.out.println(line);
1140                 line= reader.readLine();
1141             }
1142         }
1143         catch (IOException e)
1144         {
1145             System.out.println("An error occurred while reading the file.");
1146         }
1147     }
1148 }
1149
1150 /*QB 305
1151 Write a program to create directories (/home/abc/bcd/def/ghi/jkl) in the home directory
1152 /home/abc and list the files and directories showing file/directory,file size.
1153 Read-writeexecute permissions. */
1154 import java.io.File;
1155 import java.io.FileWriter;
1156 import java.io.IOException;
1157
1158 public class DirectoryCreatorAndLister {
1159     public static void main(String[] args) {
1160         String baseDir = "/home/abc";
1161         String nestedDirs = "bcd/def/ghi/jkl";
1162
1163         String fullPath = baseDir + "/" + nestedDirs;
1164
1165         // Create directories if they don't exist
1166         File directory = new File(fullPath);
1167         if (!directory.exists()) {
1168             boolean success = directory.mkdirs();
1169             if (!success) {
1170                 System.out.println("Failed to create directories.");
1171                 return;
1172             }
1173         }
1174
1175         // List files and directories with permissions
1176         System.out.println("List of files and directories with permissions:");
1177         permissioncheck(directory);
1178     }
1179
1180     static void permissioncheck(File directory) {
1181         File[] files = directory.listFiles();
1182         if (files != null) {
1183             for (File file : files) {
1184                 String name = file.getName();
1185                 String type = file.isFile() ? "File" : "Directory";
1186                 String permissions = "Read: " + file.canRead() + ", Write: " + file.

```

```

1185         canWrite() + ", Execute: " + file.canExecute();
1186         System.out.println(name + " - " + file.getAbsolutePath() + " - " + type +
            " - " + permissions);
1187     }
1188 }
1189 }
1190 }
1191
1192 //-----
1193
1194 /*QB 306
1195 Write a program to find the sum of all the number in Number.txt file and print the
1196 result in console. */
1197
1198 import java.io.BufferedReader;
1199 import java.io.FileReader;
1200 import java.io.IOException;
1201
1202 public class SumFromFile {
1203     public static void main(String[] args) throws IOException {
1204         String fileName = "Number.txt";
1205
1206         int sum = 0;
1207         FileReader fr = new FileReader(fileName);
1208         BufferedReader br = new BufferedReader(fr);
1209         String line;
1210         while ((line = br.readLine()) != null) {
1211             int number = Integer.parseInt(line.trim());
1212             sum += number;
1213         }
1214         System.out.println("Sum of numbers in " + fileName + ": " + sum);
1215     }
1216 }
1217
1218 //-----
1219
1220 /*Qb 307
1221 Write a program to create a new file and write the number into it and sort that file
1222 content into different file.*/
1223
1224 import java.io.*;
1225 import java.util.ArrayList;
1226 import java.util.Collections;
1227
1228 public class FileSorter {
1229     public static void main(String[] args) throws IOException {
1230         // Create a new file and write numbers into it
1231         String inputFilePath = "input.txt";
1232
1233         FileWriter writer = new FileWriter(inputFilePath);
1234         BufferedWriter bufferedWriter = new BufferedWriter(writer);
1235
1236         // Add some numbers to the file
1237         bufferedWriter.write("10\n5\n8\n1\n3\n");
1238         bufferedWriter.close();
1239
1240         // Sort the content of the file and write into another file
1241         String outputFilePath = "output.txt";
1242
1243         FileReader reader = new FileReader(inputFilePath);
1244         BufferedReader bufferedReader = new BufferedReader(reader);
1245
1246         ArrayList<Integer> numbers = new ArrayList<>();
1247
1248         // Read numbers from the file and add them to the ArrayList
1249         String line = bufferedReader.readLine();
1250         while (line != null) {
1251             numbers.add(Integer.parseInt(line));
1252         }
1253     }
1254 }

```

```

1250         line = bufferedReader.readLine();
1251     }
1252     bufferedReader.close();
1253
1254     // Sort the numbers in ascending order
1255     Collections.sort(numbers);
1256
1257     // Write the sorted numbers into the output file
1258     FileWriter writer = new FileWriter(outputFilePath);
1259     BufferedWriter bufferedWriter = new BufferedWriter(writer);
1260
1261     for (int number : numbers) {
1262         bufferedWriter.write(String.valueOf(number));
1263         bufferedWriter.newLine();
1264     }
1265     bufferedWriter.close();
1266
1267     System.out.println("File sorting successful.");
1268 }
1269 }
1270
1271 //-----
1272
1273 //-----CHAP 8 CHARACTER STREAM QB SOLUTION Q325 TO Q338 -----//
1274
1275 /*QB325
1276 Write a JAVA program to read student.txt file and display the content. */
1277
1278 import java.io.BufferedReader;
1279 import java.io.FileReader;
1280 import java.io.IOException;
1281
1282 public class ReadStudentFile {
1283
1284     public static void main(String[] args) throws IOException {
1285         String filename = "student.txt";
1286
1287         BufferedReader br = new BufferedReader(new FileReader(filename));
1288
1289         //loop to display contents line-by-line
1290         String line = br.readLine();
1291         while (line != null) {
1292             System.out.println(line);
1293             line = br.readLine();
1294         }
1295
1296         /* OR USE THIS LOOP instead:
1297         String line;
1298         while ((line = br.readLine()) != null) {
1299             System.out.println(line);
1300         }
1301         */
1302
1303         br.close();
1304     }
1305 }
1306 }
1307
1308 //-----
1309
1310 /*QB 326
1311 Write a program to read the content of a file into a character array and write it into
1312 another file. Get names of the files from command line. */
1313
1314 import java.io.*;
1315
1316 public class FileCopy {
1317     public static void main(String[] args) throws IOException {
1318         // Check if the correct number of command-line arguments are provided

```

```

1318         if (args.length != 2) {
1319             System.out.println("Usage: java FileCopy <source-file> <destination-file>");
1320             return;
1321         }
1322
1323         String sourceFileName = args[0];
1324         String destinationFileName = args[1];
1325
1326         // Open the source file and read its content into a character array
1327         FileReader fileReader = new FileReader(sourceFileName);
1328         int fileSize = (int) new File(sourceFileName).length();
1329         char[] content = new char[fileSize];
1330         fileReader.read(content);
1331         fileReader.close();
1332
1333         // Write the content from the character array to the destination file
1334         FileWriter fileWriter = new FileWriter(destinationFileName);
1335         fileWriter.write(content);
1336         fileWriter.close();
1337
1338         System.out.println("File copied successfully.");
1339     }
1340 }
1341
1342 //-----
1343 /*QB 327 AND QB 331 SAME
1344 Write a java program to search the file named the word entered as a filename from
command line; if it exists in the system then program should print the content of a file
on console. */
1345
1346 import java.io.*;
1347
1348 public class FileSearchAndRead {
1349     public static void main(String[] args) throws IOException {
1350         if (args.length != 1) {
1351             System.out.println("Usage: java FileSearchAndRead <filename>");
1352             return;
1353         }
1354
1355         String fileName = args[0];
1356         File file = new File(fileName);
1357
1358         if (!file.exists()) {
1359             System.out.println("File not found.");
1360             return;
1361         }
1362
1363         BufferedReader reader = new BufferedReader(new FileReader(file));
1364         String line;
1365         while ((line = reader.readLine()) != null) {
1366             System.out.println(line);
1367         }
1368         reader.close();
1369     }
1370 }
1371
1372 //-----
1373 /*QB 328
1374 Write a java program that read employee details and store into emp.txt text file using
file handling.*/
1375 import java.io.BufferedWriter;
1376 import java.io.FileWriter;
1377 import java.io.IOException;
1378 import java.util.Scanner;
1379
1380 public class EmployeeDetails {
1381     public static void main(String[] args) throws IOException {
1382         Scanner scanner = new Scanner(System.in);
1383

```

```

1384     System.out.println("Enter the number of employees: ");
1385     int numEmployees = scanner.nextInt();
1386     scanner.nextLine(); // Clear the newline character from the buffer
1387
1388     BufferedWriter writer = new BufferedWriter(new FileWriter("emp.txt"));
1389     for (int i = 0; i < numEmployees; i++) {
1390         System.out.println("Enter employee name: ");
1391         String name = scanner.nextLine();
1392
1393         System.out.println("Enter employee ID: ");
1394         int id = scanner.nextInt();
1395         scanner.nextLine(); // Clear the newline character from the buffer
1396
1397         System.out.println("Enter employee designation: ");
1398         String designation = scanner.nextLine();
1399
1400         System.out.println("Enter employee salary: ");
1401         double salary = scanner.nextDouble();
1402         scanner.nextLine(); // Clear the newline character from the buffer
1403
1404         // Write employee details to the file
1405         writer.write("Name: " + name);
1406         writer.newLine();
1407         writer.write("ID: " + id);
1408         writer.newLine();
1409         writer.write("Designation: " + designation);
1410         writer.newLine();
1411         writer.write("Salary: " + salary);
1412         writer.newLine();
1413         writer.newLine();
1414     }
1415     writer.close();
1416     System.out.println("Employee details written to emp.txt successfully.");
1417 }
1418 }
1419
1420 //-----
1421 /*QB 329
1422 Write a program that counts number of characters, words, and lines in a text file. */
1423 import java.io.BufferedReader;
1424 import java.io.FileReader;
1425 import java.io.IOException;
1426
1427 public class FileWordCount {
1428     public static void main(String[] args) throws IOException {
1429         if (args.length != 1) {
1430             System.out.println("Usage: java FileWordCount <file_path>");
1431             return;
1432         }
1433
1434         String filePath = args[0];
1435         int charCount = 0;
1436         int wordCount = 0;
1437         int lineCount = 0;
1438
1439         BufferedReader reader = new BufferedReader(new FileReader(filePath));
1440
1441         String line;
1442         while ((line = reader.readLine()) != null) {
1443             lineCount++;
1444             charCount += line.length();
1445             String[] words = line.trim().split("\\s+");
1446             wordCount += words.length;
1447         }
1448
1449         reader.close();
1450
1451         System.out.println("Number of characters: " + charCount);
1452         System.out.println("Number of words: " + wordCount);

```

```

1453         System.out.println("Number of lines: " + lineCount);
1454     }
1455 }
1456
1457 //-----
1458 /*QB 330
1459 Create a class called Student. Write a student manager program to manipulate the student
1460 information from files by using the BufferedReader and BufferedWriter.*/
1461 import java.io.*;
1462
1463 class Student {
1464     private String name;
1465     private int age;
1466     private String rollNumber;
1467
1468     public Student(String name, int age, String rollNumber) {
1469         this.name = name;
1470         this.age = age;
1471         this.rollNumber = rollNumber;
1472     }
1473
1474     public String getName() {
1475         return name;
1476     }
1477
1478     public int getAge() {
1479         return age;
1480     }
1481
1482     public String getRollNumber() {
1483         return rollNumber;
1484     }
1485
1486     @Override
1487     public String toString() {
1488         return "Name: " + name + ", Age: " + age + ", Roll Number: " + rollNumber;
1489     }
1490 }
1491
1492 public class StudentManager {
1493     public static void main(String[] args) throws IOException {
1494         // File paths
1495         String inputFile = "input.txt";
1496         String outputFile = "output.txt";
1497
1498         // Read student information from the input file and store in a list
1499         BufferedReader reader = new BufferedReader(new FileReader(inputFile));
1500         String line;
1501         List<Student> students = new ArrayList<>();
1502         while ((line = reader.readLine()) != null) {
1503             String[] data = line.split(",");
1504             if (data.length == 3) {
1505                 String name = data[0].trim();
1506                 int age = Integer.parseInt(data[1].trim());
1507                 String rollNumber = data[2].trim();
1508                 students.add(new Student(name, age, rollNumber));
1509             }
1510         }
1511         reader.close();
1512
1513         // Write student information to the output file
1514         BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile));
1515         for (Student student : students) {
1516             writer.write(student.toString());
1517             writer.newLine();
1518         }
1519         writer.close();
1520
1521         // Print the student information to the console

```

```

1521         for (Student student : students) {
1522             System.out.println(student);
1523         }
1524     }
1525 }
1526
1527 //-----
1528
1529 /*QB 331    (SAME AS 327)
1530 Write a java program to search the file named the word entered as a filename from
commandline: If it exists in the system the program should print the contents of the
file on console */
1531
1532 import java.io.BufferedReader;
1533 import java.io.FileReader;
1534 import java.io.IOException;
1535
1536 public class FileSearchAndRead {
1537     public static void main(String[] args) throws IOException {
1538         if (args.length != 1) {
1539             System.out.println("Usage: java FileSearchAndRead <filename>");
1540             return;
1541         }
1542
1543         String filename = args[0];
1544         BufferedReader reader = new BufferedReader(new FileReader(filename));
1545         String line;
1546
1547         while ((line = reader.readLine()) != null) {
1548             System.out.println(line);
1549         }
1550
1551         reader.close();
1552     }
1553 }
1554
1555 //-----
1556 /*QB 332
1557 Write a Java program to copy content of file1.txt to file2.txt using Java file handling
*/
1558
1559 import java.io.*;
1560
1561 public class FileCopy {
1562     public static void main(String[] args) throws IOException {
1563         // Provide the paths for file1.txt and file2.txt
1564         String sourcePath = "file1.txt";
1565         String destinationPath = "file2.txt";
1566
1567         // Open the input and output streams
1568         FileInputStream inputStream = new FileInputStream(sourcePath);
1569         FileOutputStream outputStream = new FileOutputStream(destinationPath);
1570
1571         // Copy the content from file1 to file2
1572         int data;
1573         while ((data = inputStream.read()) != -1) {
1574             outputStream.write(data);
1575         }
1576
1577         // Close the streams
1578         inputStream.close();
1579         outputStream.close();
1580
1581         System.out.println("Content of file1.txt has been copied to file2.txt.");
1582     }
1583 }
1584
1585 //-----
1586 /*QB 333

```

```

1587 Write an application that reads a file and counts the number of occurrences of digit 5.
1588 Supply the file name as a command-line argument*/
1589
1590
1591 import java.io.*;
1592
1593 class CountDigit5Occurrences
1594 {
1595     public static void main(String[] args) throws Exception
1596     {
1597         if (args.length != 1)
1598         {
1599             System.err.println("Usage: java CountDigit5Occurrences <filename>");
1600             return;
1601         }
1602
1603         String filename = args[0];
1604         int count = 0;
1605         BufferedReader reader = new BufferedReader(new FileReader(filename));
1606         String line = reader.readLine();
1607         while (line != null)
1608         {
1609             char ch[] = line.toCharArray();
1610             for (char c : ch)
1611             {
1612                 if (c == '5')
1613                 {
1614                     count++;
1615                 }
1616             }
1617             line = reader.readLine();
1618         }
1619         System.out.println("Number of occurrences of digit 5 in the file: " + count);
1620     }
1621 }
1622
1623 //-----
1624
1625 /*QB 334
1626 Write a program to replace all "word1" by "word2" to a file and
1627 display the no. of replacement.*/
1628
1629 import java.io.*;
1630 import java.util.*;
1631
1632 class FileReplace
1633 {
1634     public static void main(String[] args) throws IOException
1635     {
1636         File f1 = new File("G:\\bb.txt"); // read from this and replace
1637         File f2 = new File("G:\\bbb.txt"); // write to this after replace
1638         FileWriter fw1 = new FileWriter(f1);
1639         fw1.write("Hi baby Hi Hi its too High");
1640         fw1.close();
1641         Scanner sc = new Scanner(System.in);
1642         System.out.println("Enter Word 1");
1643         String word1 = sc.next();
1644         System.out.println("Enter Word 2");
1645         String word2 = sc.next();
1646         FileWriter fw2 = new FileWriter(f2);
1647         FileReader fr1 = new FileReader(f1);
1648         BufferedReader br1 = new BufferedReader(fr1); //
1649         String rep = "";
1650         String s="";
1651         int count = 0 ;
1652         while((s=br1.readLine())!=null)
1653         {
1654             String s1[] = s.split(" ");
1655             for(String s2 : s1)
1656             {
1657                 if(s2.equals(word1))

```

```

1655         {
1656             count++;
1657             s2=word2;
1658         }
1659         rep = rep+s2+" ";
1660     }
1661     System.out.println("Count of Replacement = "+count);
1662     fw2.write(rep+"\n");
1663 }
1664 fw2.close();
1665 }
1666 }
1667 //-----
1668
1669 /*QB 335
1670 Write a program to count the total no. of chars, words, lines, alphabets,
1671 digits, white spaces in a given file. */
1672 import java.io.*;
1673
1674 class FileAnalyzer {
1675
1676     public static void main(String[] args)
1677     {
1678         if (args.length != 1) {
1679             System.err.println("Usage: java FileAnalyzer <filename>");
1680             return;
1681         }
1682
1683         String filename = args[0];
1684
1685         try {
1686             File file = new File(filename);
1687             BufferedReader reader = new BufferedReader(new FileReader(file));
1688
1689             int charCount = 0;
1690             int wordCount = 0;
1691             int lineCount = 0;
1692             int alphabetCount = 0;
1693             int digitCount = 0;
1694             int whitespaceCount = 0;
1695
1696             String line;
1697             while ((line = reader.readLine()) != null) {
1698                 charCount += line.length();
1699                 wordCount += countWords(line);
1700                 alphabetCount += countAlphabets(line);
1701                 digitCount += countDigits(line);
1702                 whitespaceCount += countWhitespaces(line);
1703                 lineCount++;
1704             }
1705
1706             reader.close();
1707
1708             System.out.println("Total characters: " + charCount);
1709             System.out.println("Total words: " + wordCount);
1710             System.out.println("Total lines: " + lineCount);
1711             System.out.println("Total alphabets: " + alphabetCount);
1712             System.out.println("Total digits: " + digitCount);
1713             System.out.println("Total white spaces: " + whitespaceCount);
1714         } catch (IOException e) {
1715             e.printStackTrace();
1716         }
1717     }
1718
1719     static int countWords(String line) {
1720         String[] words = line.split(" ");
1721         return words.length;
1722     }
1723 }

```

```

1724     static int countAlphabets(String line) {
1725         int count = 0;
1726         for (char c : line.toCharArray()) {
1727             if (Character.isLetter(c)) {
1728                 count++;
1729             }
1730         }
1731         return count;
1732     }
1733
1734     static int countDigits(String line) {
1735         int count = 0;
1736         for (char c : line.toCharArray()) {
1737             if (Character.isDigit(c)) {
1738                 count++;
1739             }
1740         }
1741         return count;
1742     }
1743
1744     static int countWhitespaces(String line) {
1745         int count = 0;
1746         for (char c : line.toCharArray()) {
1747             if (Character.isWhitespace(c)) {
1748                 count++;
1749             }
1750         }
1751         return count;
1752     }
1753 }
1754
1755 //-----
1756
1757 /*QB 336
1758 Write a program to display the bytes of a file in reverse sequence. Provide the name of
1759 the file as a command line argument. */
1760
1761 import java.io.*;
1762
1763 public class ReverseFileBytes {
1764     public static void main(String[] args) throws IOException {
1765         if (args.length != 1) {
1766             System.out.println("Usage: java ReverseFileBytes <filename>");
1767             return;
1768         }
1769
1770         File file = new File(args[0]);
1771         if (!file.exists() || !file.isFile()) {
1772             System.out.println("File not found or is not a regular file.");
1773             return;
1774         }
1775
1776         RandomAccessFile raf = new RandomAccessFile(file, "r");
1777         long fileSize = raf.length();
1778
1779         for (long position = fileSize - 1; position >= 0; position--) {
1780             raf.seek(position);
1781             byte byteValue = raf.readByte();
1782             System.out.print(byteValue);
1783         }
1784         raf.close();
1785     }
1786 }
1787
1788 //-----
1789
1790 /*QB 337
1791 Write a program to sort the one file numbers to another.for example
1792 one file contain the unsorted number separated by line and write the another

```

```

1792 file with sorted number. */
1793
1794 import java.io.*;
1795 import java.util.ArrayList;
1796 import java.util.Collections;
1797
1798 class SortNumbersFromFile
1799 {
1800     public static void main(String[] args) {
1801         if (args.length != 2) {
1802             System.err.println("Usage: java SortNumbersFromFile <inputFile><outputFile>"
1803                 );
1804             return;
1805         }
1806
1807         String inputFile = args[0];
1808         String outputFile = args[1];
1809
1810         ArrayList<Integer> numbers = new ArrayList<>();
1811
1812         try {
1813             BufferedReader reader = new BufferedReader(new FileReader(inputFile));
1814
1815             String line;
1816             while ((line = reader.readLine()) != null) {
1817                 try {
1818                     int number = Integer.parseInt(line.trim());
1819                     numbers.add(number);
1820                 } catch (NumberFormatException e) {
1821                     // Ignore non-integer lines in the input file
1822                 }
1823             }
1824
1825             reader.close();
1826
1827             Collections.sort(numbers);
1828
1829             BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile));
1830
1831             for (int num : numbers) {
1832                 writer.write(String.valueOf(num));
1833                 writer.newLine();
1834             }
1835
1836             writer.close();
1837
1838             System.out.println("Numbers sorted and written to " + outputFile);
1839         } catch (IOException e) {
1840             e.printStackTrace();
1841         }
1842     }
1843     //-----
1844     /*QB338
1845     Write a java program which read numbers from number.txt file and store even number to
1846     even.txt and odd number into odd.txt file. */
1847
1848 import java.io.*;
1849
1850 public class FileNumberSorter {
1851     public static void main(String[] args) throws IOException {
1852         BufferedReader reader = new BufferedReader(new FileReader("number.txt"));
1853         BufferedWriter evenWriter = new BufferedWriter(new FileWriter("even.txt"));
1854         BufferedWriter oddWriter = new BufferedWriter(new FileWriter("odd.txt"));
1855
1856         String line;
1857         while ((line = reader.readLine()) != null) {
1858             int num = Integer.parseInt(line);
1859             if (num % 2 == 0) {

```

```
1859         evenWriter.write(line);
1860         evenWriter.newLine();
1861     } else {
1862         oddWriter.write(line);
1863         oddWriter.newLine();
1864     }
1865 }
1866
1867 reader.close();
1868 evenWriter.close();
1869 oddWriter.close();
1870 }
1871 }
```